

DIGITAL TWIN CATALOG

# Railway Deployment Guide

Step-by-step deployment of the database, API and backup job on Railway, with checks and troubleshooting.

|            |                                                                                                       |
|------------|-------------------------------------------------------------------------------------------------------|
| Project    | Digital Twin Catalog (NARAGA 3D Catalog), reusable 3DCityDB template                                  |
| Document   | Railway Guide                                                                                         |
| Version    | 0.1 draft for review, 4 October 2026                                                                  |
| Author     | Fairuz Akmal Pradana                                                                                  |
| Repository | <a href="https://github.com/fakmalpradana/naraga-twin-db">github.com/fakmalpradana/naraga-twin-db</a> |

# **Contents**

**Introduction**

**Before you start**

**Step 1. Create the project**

**Step 2. Create the db service**

**Step 3. Create the api service**

**Step 4. Verify the deployment**

**Step 5. Backups (recommended before loading data)**

**Step 6. Load the OIKN data**

**Step 7. Publish tiles and test in the viewer**

**Step 8. Turn on editing and add users (14 to 16 October)**

**Operations**

**Troubleshooting**

**Checklist**

# Introduction

---

This guide deploys the catalog on Railway step by step: a **db** service (3DCityDB v5 with a volume), an **api** service (Django: API, admin, viewer) and an optional **backup** job. Every step ends with a check, so you know it worked before moving on.

Read this first:

- **Railway Config as Code ( `railway.toml` ) is deprecated and stops being read on 1 December 2026.** For that reason the repository has no `railway.toml` . Everything is set in the Railway dashboard and in service variables, which keeps working. A root `railway.toml` would also apply to every service built from the same repository folder.
- The first start of the db volume fixes the coordinate system ( `SRID` , `HEIGHT_EPSG` ). Decide them before step 3. Changing them later means recreating the volume.
- Items marked *verify* were not testable without a Railway account and are the first things to check if something fails.
- Already tested locally on 4 October 2026 with Docker, imitating Railway: the db image with `PGDATA` in a sub-folder of an empty volume initialises and keeps its data after a restart; the api image starts from that empty database in production mode ( `DEBUG=0` , `PORT` set, allowed hosts) and applies all migrations, creates the login roles and the first admin, answers the health check, and rejects an unknown host with 400. What only Railway can confirm: private-network name resolution, volume behaviour on its platform, the dashboard field names and the cost.

| Service           | Built from               | Volume                                           | Public address |
|-------------------|--------------------------|--------------------------------------------------|----------------|
| db                | deploy/db/Dockerfile     | yes, at<br><code>/var/lib/postgresql/data</code> | no             |
| api               | api/Dockerfile           | no                                               | yes            |
| backup (optional) | deploy/backup/Dockerfile | no                                               | no             |

## Before you start

| Need                                      | Detail                                                                                                                                               |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Railway account on a paid plan            | The trial volume is 0.5 GB, Hobby 5 GB, Pro 50 GB (growable to 1 TB). A volume can be enlarged, never shrunk. Size it for your LOD data plus a dump. |
| GitHub repository connected to Railway    | <code>fakmalpradana/naraga-twin-db</code>                                                                                                            |
| Coordinate system                         | Source CRS of the OIKN data, for example <code>32750</code> (WGS 84 / UTM 50S) with <code>HEIGHT_EPSG=4979</code>                                    |
| Secrets                                   | Generate with <code>scripts/gen_secrets.sh</code> (prints random values; copy them into Railway, never into git)                                     |
| Backup bucket (optional now, recommended) | Any S3-compatible bucket (for example Cloudflare R2 or Backblaze B2) with an access key                                                              |
| Domain (optional)                         | Railway gives a <code>*.up.railway.app</code> address; a custom domain can be added later                                                            |

Generate the secrets on your machine:

```
scripts/gen_secrets.sh
```

Keep the output in a password manager. You will paste `POSTGRES_PASSWORD` , `APP_DB_PASSWORD` , `API_DB_PASSWORD` , `ADMIN_PASSWORD` and `SECRET_KEY` .

## Step 1. Create the project

---

1. In Railway choose **New Project**, then **Empty Project**. Name it `naraga-twin`.
2. Open **Settings** of the project and confirm the plan and region.

Check: an empty canvas is shown.

## Step 2. Create the db service

---

1. **New**, then **GitHub Repo**, choose `naraga-twin-db`. Railway creates a service and starts a build that will use the wrong Dockerfile. Stop it by continuing with the next sub-steps before it finishes; a failed first deploy is harmless.
2. Rename the service to `db` (Settings, Service Name). The name matters: other services reach it as `db.railway.internal`.
3. **Settings, Build**: set the Dockerfile path to `deploy/db/Dockerfile`. If the dashboard has no such field, add the variable `RAILWAY_DOCKERFILE_PATH=deploy/db/Dockerfile`.
4. **Variables** (add as a raw editor paste):

```
POSTGRES_PASSWORD=<from gen_secrets>
SRID=32750
HEIGHT_EPSG=4979
```

5. **Volumes**: add a volume to the service and mount it at `/var/lib/postgresql/data`. The Dockerfile already sets `PGDATA` to a sub-folder, because a fresh volume contains a `lost+found` folder that PostgreSQL refuses to initialise over.
6. Do **not** generate a public domain or TCP proxy for `db`.
7. **Deploy**.

Check: in **Deployments, View logs** the db log ends with a line like "database system is ready to accept connections" **twice** (the image restarts once after initialising 3DCityDB). The first start can take a few minutes.

Notes: a service with a volume cannot run two deployments at once, so every redeploy of `db` has a short downtime. The image is built for amd64, which is what Railway runs.

## Step 3. Create the api service

1. **New, GitHub Repo**, `naraga-twin-db` again. Rename it to `api`.
2. **Settings, Build**: Dockerfile path `api/Dockerfile` (or variable `RAILWAY_DOCKERFILE_PATH=api/Dockerfile`). Leave the root directory at the repository root, because the Dockerfile copies `db/`, `api/`, `viewer/` and `client/`.
3. **Settings, Deploy**: healthcheck path `/api/v1/health`, healthcheck timeout `180`, restart policy `On Failure`.
4. **Variables**:

```
PROJECT=oikn
DATABASE_URL=postgres://postgres:${db.POSTGRES_PASSWORD}@db.railway.internal:5432/postgres?
sslmode=disable
APP_DB_PASSWORD=<from gen_secrets>
API_DB_PASSWORD=<from gen_secrets>
SECRET_KEY=<from gen_secrets>
DEBUG=0
ADMIN_USERNAME=admin
ADMIN_PASSWORD=<from gen_secrets>
ADMIN_EDIT_ENABLED=0
ALLOWED_HOSTS=<set in step 4>
CSRF_TRUSTED_ORIGINS=<set in step 4>
ALLOWED_ORIGINS=<frontend addresses, set in step 4>
```

`${db.POSTGRES_PASSWORD}` is Railway's reference to a variable of the `db` service, so the password lives in one place.

5. **Settings, Networking, Generate Domain**. Note the address, for example `api-production-1a2b.up.railway.app`.
6. Go back to **Variables** and set the three values that need the address:

```
ALLOWED_HOSTS=api-production-1a2b.up.railway.app
CSRF_TRUSTED_ORIGINS=https://api-production-1a2b.up.railway.app
ALLOWED_ORIGINS=https://your-frontend.example.com
```

`ALLOWED_ORIGINS` is the CORS allow-list for the Cesium frontend; separate several with commas. The viewer served by the api itself does not need to be listed because it uses the same address.

7. **Deploy** (or **Redeploy**).

What happens on every start (`api/entrypoint.sh`, all idempotent): wait for the database, `dbmate up` (migrations), create the login roles `twin_app` and `twin_api`, run `db/seeds/oikn/*.sql`, Django migrate, create groups `viewer`, `editor`, `admin` and the first admin user, then start gunicorn.

Check:

```
curl https://api-production-1a2b.up.railway.app/api/v1/health
```

should print `{"status": "ok", "database": "ok"}`. If the deploy fails the log shows which step stopped; see Troubleshooting.

*verify:* Railway private networking resolves `db.railway.internal` over IPv6 as well as IPv4. The PostgreSQL image listens on all addresses, so this should work; if the api log says "connection refused", redeploy `db` and then `api`.



## Step 4. Verify the deployment

---

1. Open `https://<api address>/admin/` and sign in with `ADMIN_USERNAME` and `ADMIN_PASSWORD`. You should see **Datasets, Layers, Tilesets, Import jobs** and **Inventory**; the inventory lists three draft layers from the OIKN seed.
2. Open `https://<api address>/viewer/`. The layer lists load from the API; layers show "no active tileset yet" until you publish one.
3. Run the Postman collection against Railway from your machine:

```
make deploy-check URL=https://<api address>
```

or in Postman choose the `railway` environment and set `baseUrl`.

Check: 0 failed assertions. The tests for features and tilesets skip themselves until data exists.

## Step 5. Backups (recommended before loading data)

---

1. **New, GitHub Repo**, `naraga-twin-db`; rename the service to `backup`.
2. **Settings, Build**: Dockerfile path `deploy/backup/Dockerfile`.
3. **Settings, Deploy**: cron schedule `0 18 * * *` (18:00 UTC, 01:00 WIB) and restart policy `Never`.
4. **Variables**:

```
DATABASE_URL=postgres://postgres:${db.POSTGRES_PASSWORD}@db.railway.internal:5432/postgres?
sslmode=disable
BACKUP_BUCKET=<bucket name>
AWS_ACCESS_KEY_ID=<key>
AWS_SECRET_ACCESS_KEY=<secret>
AWS_ENDPOINT_URL=<only for non-AWS providers, for example the R2 endpoint>
AWS_DEFAULT_REGION=auto
```

5. Deploy once and read the log: it prints the dump size and the upload result. Set a lifecycle rule on the bucket to delete files older than 30 days.
6. **Restore test (do this once, before loading real data)**: download a dump and follow the restore procedure in the Operations Runbook. A backup that was never restored is not a backup.

## Step 6. Load the OIKN data

---

Import with `citydb-tool` from your machine through a **temporary** TCP proxy. Large files over the internet are slow; for very large data consider running the import in a Railway one-off container or on a machine in the same region.

1. In `db`, **Settings, Networking, TCP Proxy**, add port `5432`. Railway shows a host and port, for example `roundhouse.proxy.rlwy.net:12345`.
2. Run the import wrapper with the remote address:

```
scripts/import.sh USER=fairuz FILE=/data/oikn_lod1.gml DATASET=oikn THEME=building LOD=1 \
  REMOTE_HOST=roundhouse.proxy.rlwy.net REMOTE_PORT=12345 REMOTE_PASSWORD=<POSTGRES_PASSWORD>
```

3. **Remove the TCP proxy** from `db` when finished.
4. Validate with `val3dity`, record the result and move the layer to `validated` as described in the Operations Runbook.

Check: the inventory page shows the object count and bounding box of the layer, and `GET /api/v1/datasets/oikn/stats` returns counts.

## Step 7. Publish tiles and test in the viewer

---

1. Convert with convertwin and upload to Cesium ion; note the asset id.
2. In the admin, add a Tileset to the layer ( `cesium_ion` , asset id, status Ready), use the action *Make this build the active one*, then set the layer to Published.
3. Open `/viewer/` , paste your Cesium ion token (kept in the browser only), choose the layer and load it. Click a building: the side panel shows its attributes from the database. Whether convertwin keeps the `gml:id` in the tiles is still to be verified with real tiles.

## Step 8. Turn on editing and add users (14 to 16 October)

---

1. In `api` Variables set `ADMIN_EDIT_ENABLED=1` and redeploy.
2. In **Admin, Users** add one account per person and put them in group `viewer`, `editor` or `admin`.  
Share the admin address, not the passwords of others.
3. Check that an edit by a user appears under that record's **History** with their name.

# Operations

---

| Task                            | How                                                                                                                                                        |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| View logs                       | Service, Deployments, View logs                                                                                                                            |
| Roll back the api               | Deployments, pick the previous successful deployment, Redeploy. Migrations only move forward, so a rollback after a schema change needs a matching restore |
| Change a variable               | Variables, edit, Railway redeploys the service                                                                                                             |
| Enlarge the volume              | db , Volumes, resize (live, cannot shrink)                                                                                                                 |
| Reach the DB for a one-off task | Enable the TCP proxy, work, remove it                                                                                                                      |
| Upgrade 3DCityDB                | See the Operations Runbook; change the tag in <code>deploy/db/Dockerfile</code>                                                                            |
| Deleted a volume by mistake     | Railway keeps it for 48 hours; restore it from the dashboard                                                                                               |
| Cost                            | Pay per usage: the db is the main cost (memory and volume). Check Railway usage after the first week                                                       |

# Troubleshooting

| Symptom                                                             | Likely cause and fix                                                                                                                          |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| api build fails: file not found <code>api/Dockerfile</code>         | Build settings point at the wrong path or the root directory is not the repository root                                                       |
| db deploy loops with "directory exists but is not empty"            | <code>PGDATA</code> sub-folder missing: the <code>db</code> service must use <code>deploy/db/Dockerfile</code> , not the api Dockerfile       |
| db log shows "role/database does not exist" for <code>citydb</code> | The image did not initialise: <code>SRID</code> and <code>HEIGHT_EPSG</code> variables missing at first start. Delete the volume and redeploy |
| api log "waiting for database" forever                              | Service not named <code>db</code> , wrong <code>DATABASE_URL</code> , or db still initialising                                                |
| api log "APP_DB_PASSWORD required"                                  | Variables not set                                                                                                                             |
| 400 Bad Request on the api address                                  | The address is missing from <code>ALLOWED_HOSTS</code>                                                                                        |
| Admin login: CSRF verification failed                               | The address is missing from <code>CSRF_TRUSTED_ORIGINS</code> (must start with <code>https://</code> )                                        |
| Browser console CORS error from the frontend                        | The frontend address is missing from <code>ALLOWED_ORIGINS</code>                                                                             |
| Viewer shows "Cannot reach the API"                                 | Wrong API address field or CORS                                                                                                               |
| Import over the TCP proxy fails to connect                          | Proxy removed, or wrong host or port; the proxy shows a different port from 5432                                                              |
| <code>make deploy-check</code> failures on features                 | Expected until data is imported; the tests skip when no data exists                                                                           |

# Checklist

---

- ☐ Plan and volume size chosen, `SRID` and `HEIGHT_EPSG` decided
- ☐ Secrets generated and stored
- ☐ `db` healthy, volume mounted, no public address
- ☐ `api` healthy, public domain, health check green
- ☐ Admin login works; inventory lists the seed layers
- ☐ Postman `railway` environment passes
- ☐ Backup job ran once and a dump was restored successfully
- ☐ TCP proxy removed after the import
- ☐ `ADMIN_EDIT_ENABLED=1` only when editors are ready